16 Amortized Analysis

Imagine that you join Buff's Gym. Buff charges a membership fee of \$60 per month, plus \$3 for every time you use the gym. Because you are disciplined, you visit Buff's Gym every day during the month of November. On top of the \$60 monthly charge for November, you pay another $3 \times \$30 = \$90$ that month. Although you can think of your fees as a flat fee of \$60 and another \$90 in daily fees, you can think about it in another way. All together, you pay \$150 over 30 days, or an average of \$5 per day. When you look at your fees in this way, you are ***amortizing*** the monthly fee over the 30 days of the month, spreading it out at \$2 per day.

You can do the same thing when you analyze running times. In an ***amortized analysis***, you average the time required to perform a sequence of data-structure operations over all the operations performed. With amortized analysis, you show that if you average over a sequence of operations, then the average cost of an operation is small, even though a single operation within the sequence might be expensive. Amortized analysis differs from average-case analysis in that probability is not involved. An amortized analysis guarantees the *average performance of each operation in the worst case*.

The first three sections of this chapter cover the three most common techniques used in amortized analysis. Section 16.1 starts with aggregate analysis, in which you determine an upper bound $T(n)$ on the total cost of a sequence of n operations. The average cost per operation